INDUSTRIAL INTERNSHIP REPORT

ONLINE SUMMER INTERNSHIP IN PYTHON BY AICTE

PROJECT TITLE
File Organizer: A Python GUI Application

Submitted by

MOUNA BHAT

Third Year, Computer Science and Engineering

Srinivas Institute of Technology, Valachil, Mangaluru

Internship: Online Summer Internship in Python by AICTE

Programme report format: upskill Campus / The IoT Academy / UCT

Academic Year: 2026–2027

Prepared with reference to the project repository and supplied internship report sample.

EXECUTIVE SUMMARY

This report describes the Python project **File Organizer**, a desktop application that automatically sorts files into category folders based on their file types. The project is intended to reduce the effort required to manually arrange files in a selected directory and make folders easier to navigate.

The public GitHub repository describes a simple graphical interface that lets the user select a folder using a Browse button. The application identifies file types, creates category folders, moves files into the relevant categories, avoids overwriting duplicate file names and displays the number of files organized. The documented categories are Images, Documents, Videos, Audio, Archives, Code and Others.

The project uses Python, Tkinter, the OS module and the Shutil module. Its repository contains app.py, organizer.py, README.md and requirements.txt. This report follows the structure of the supplied internship sample and distinguishes documented functionality from recommended future improvements. It does not claim performance measurements that are not recorded in the repository.

Project at a glance

Item	Details
Project	File Organizer
Programming language	Python
GUI toolkit	Tkinter
File operations	OS and Shutil modules
Main purpose	Sort files into folders by file type
Repository	https://github.com/bhatmouna829-cloud/File-Organizer

TABLE OF CONTENTS

Section	Page
1. Preface	3
2. Introduction	4
2.1 Internship context	4
2.2 Project overview	4
2.3 Objectives	5
2.4 References and glossary	5
3. Problem Statement	6
4. Existing and Proposed Solution	7
4.1 Repository and code links	7
5. Proposed Design / Model	8
5.1 High-level workflow	8
5.2 Components and interfaces	9
6. Performance Test	10
6.1 Test plan / test cases	10
6.2 Test procedure and outcome	11
7. My Learnings	12
8. Future Work Scope	13

1. PREFACE

An internship provides an opportunity to apply programming concepts to a practical problem and gain experience in designing, implementing and documenting software. The Online Summer Internship in Python by AICTE is the context for this report, which focuses on the File Organizer project.

The project addresses a common computer-use problem: files of different types can accumulate in a single folder, making it harder to locate documents, images, videos and other items. File Organizer automates the process of sorting files into category folders based on their file types.

The application combines a graphical user interface with file-system operations. The user chooses a folder, the program identifies file types, creates suitable category folders and moves files into them. It also aims to avoid overwriting files with duplicate names and displays a count of organized files.

The project helped strengthen my understanding of Python modules, GUI development, file paths and file operations. It also demonstrated the importance of testing with different file types and handling file conflicts safely.

I am thankful to the internship programme organisers, my institution and the learning resources that supported this work. My advice to juniors is to break projects into smaller functions, test them with sample data first and document the setup and behaviour clearly.

2. INTRODUCTION

Python includes standard-library modules for interacting with the operating system, working with file paths and moving files. Tkinter can be used to build a graphical interface, allowing a user to interact with a program through buttons and windows rather than typing every command. File Organizer applies these capabilities to routine file management.

2.1 Internship context

The supplied instructions identify the programme as the Online Summer Internship in Python by AICTE and require students to upload project source code and a PDF report to a GitHub repository named upskillcampus. The report must include working links to the code and report files. The existing File Organizer project repository is available at the link provided in this report; its repository name is File-Organizer, so the required submission repository naming rule may need to be handled separately.

2.2 Project overview

File Organizer is a Python desktop application that organizes files in a selected folder into categories according to file type. It offers a Browse control for folder selection and automates the categorization and movement process. The application is designed to reduce repetitive manual sorting and produce a clearer folder structure.

2.3 Objectives

- Build a desktop file-organization application using Python.
- Provide a simple graphical interface for selecting a folder.
- Identify files according to their types and extensions.
- Create folders for relevant file categories.
- Move files into the appropriate category folders.
- Avoid overwriting files when duplicate names are encountered.
- Display the number of files organized and document the application.

2.4 References

[1] File Organizer GitHub repository: https://github.com/bhatmouna829-cloud/File-Organizer

[2] Python documentation: https://docs.python.org/3/ (language and standard-library reference).

[3] Tkinter documentation: https://docs.python.org/3/library/tkinter.html (GUI toolkit reference).

[4] Internship sample report: Sample_InternshipReport_USC_UCT.docx, supplied by the user.

2.5 Glossary

Term	Meaning
GUI	Graphical User Interface; visual controls used to interact with an application.
Tkinter	Python interface to the Tk GUI toolkit.
File extension	Suffix such as .pdf or .jpg that commonly indicates a file format.
Directory	A folder used to store files and other folders.
OS module	Python standard-library module for interacting with operating-system features.
Shutil	Python standard-library module that provides high-level file operations.

3. PROBLEM STATEMENT

When many files are saved in one folder, locating a particular item can become inconvenient. Manual sorting requires the user to inspect files one by one, decide which category each belongs to, create folders and move the files. Repeating this process takes time and may lead to inconsistent folder structures.

The problem addressed by File Organizer is to automate file categorization and movement for a folder selected by the user. The application should recognize file types, group them into predefined categories, create destination folders when required and avoid overwriting files that share a name.

3.1 Main challenges

- Selecting and accessing the correct folder from the graphical interface.
- Recognizing different file types and mapping them to categories.
- Creating destination folders only when needed.
- Handling duplicate file names without unintentionally replacing existing files.
- Managing file-system errors, permissions and unsupported cases.
- Giving the user a clear indication of the result of the organization process.

3.2 Scope

The documented scope is a desktop application that organizes files in a selected directory by file type. The repository README documents the Browse function, category folders, duplicate-name protection and a count of organized files. The report does not claim recursive processing, undo, preview mode, cloud integration or other features that are not documented in the repository.

Expected outcome

A simpler file-management workflow in which a user selects a folder and the application sorts its files into categories such as Images, Documents, Videos, Audio, Archives, Code and Others.

4. EXISTING AND PROPOSED SOLUTION

4.1 Existing approach

The basic alternative is to organize files manually by creating folders and moving each file into the right location. This approach gives the user direct control but can become repetitive when a folder contains many different file types. The proposed application automates the repetitive categorization and movement steps.

Approach	Limitation or benefit
Manual organization	Requires repeated inspection, folder creation and file movement.
File Organizer	Automates categorization and movement through a simple GUI; behaviour still depends on file-type rules and file-system permissions.

4.2 Proposed solution

The proposed solution is a Python desktop application with a Tkinter graphical interface. It allows the user to select a folder, identifies file types, creates category folders, moves files into the corresponding locations, prevents duplicate names from being overwritten and reports the number of files organized.

- Choose a folder using the Browse button.
- Scan or inspect the selected folder's files.
- Identify file types and map them to categories.
- Create the relevant category folders.
- Move files to the matching folders while handling duplicate names.
- Display the number of files organized.

4.3 Value addition

The application reduces repetitive manual file sorting and provides a consistent category structure. Its GUI makes the process accessible without requiring the user to enter file paths in a command line. Duplicate-name handling also reduces the risk of accidentally overwriting an existing file.

4.4 GitHub code submission

Project repository: https://github.com/bhatmouna829-cloud/File-Organizer

README: Open README.md

Application code: app.py

Organizer logic: organizer.py

Final report link: Add the PDF to the required GitHub repository and replace this line with the resulting working file URL. The final link cannot exist until the report is uploaded.

5. PROPOSED DESIGN / MODEL

The application follows a straightforward workflow: the user selects a folder, the application examines the files, categorizes them, creates the required destination folders and moves each file to its category. A result count is then displayed to the user.

5.1 High-level workflow

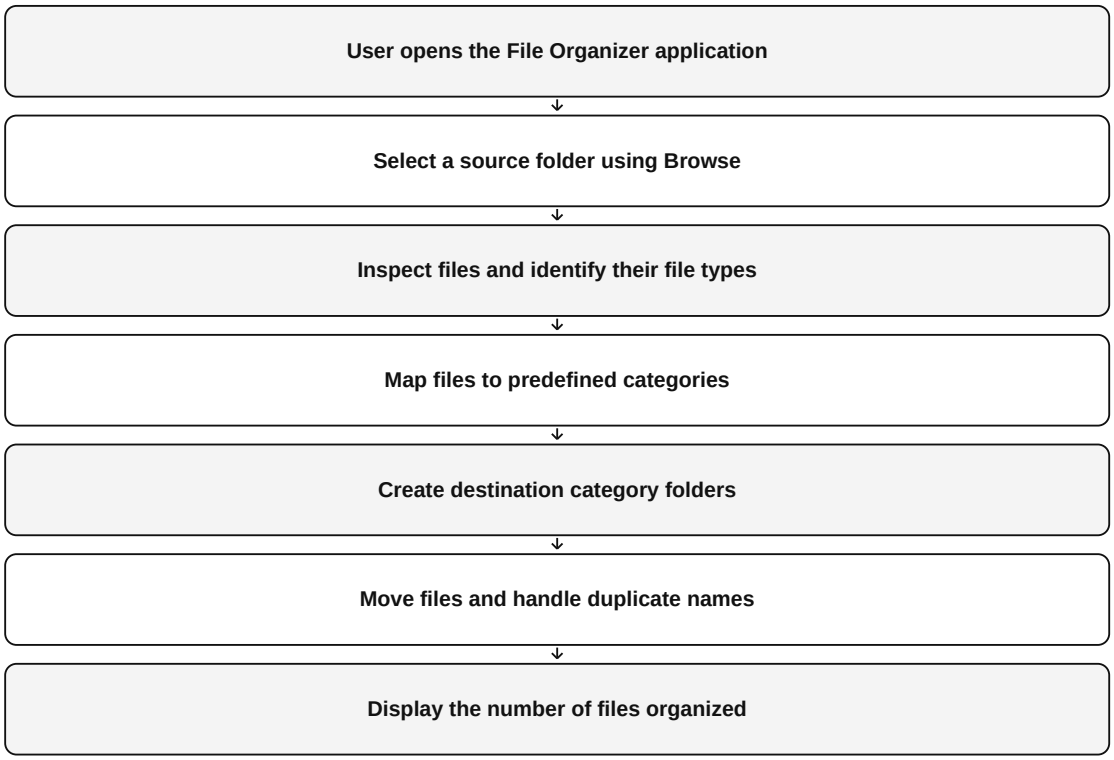

Figure 1. High-level workflow of the File Organizer application.

5.2 Main components

File / component	Purpose
app.py	Application entry point and graphical user interface, as indicated by the repository structure.
organizer.py	File-organization logic, including categorization and movement operations.
README.md	Project description, documented features and file structure.
requirements.txt	Dependency list for setting up the project environment.
Tkinter	Provides the desktop graphical interface.
OS and Shutil	Support file-system interaction and high-level file operations.

5.3 File categories and interface

Category	Examples (illustrative)
Images	Image files such as JPG or PNG.
Documents	PDFs and office/text documents.
Videos	Video files.
Audio	Audio files.
Archives	Compressed archive files.
Code	Source-code files.
Others	Files not matched to the other configured categories.

The exact supported extensions are determined by the implementation. The examples above illustrate category meanings and are not a complete extension list.

6. PERFORMANCE TEST

Testing should verify that the application can select a folder, categorize files, create destination folders, move files and handle duplicate names correctly. The repository documents the expected functionality, but it does not provide numerical benchmark results. Accordingly, this section describes functional test cases rather than inventing speed or accuracy measurements.

6.1 Test plan / test cases

ID	Test case	Expected result
T1	Launch the application.	The GUI opens and provides folder selection.
T2	Select a valid folder.	The selected folder is accepted for organization.
T3	Use a folder containing multiple known file types.	Files are sorted into their configured categories.
T4	Include files from different documented categories.	Category folders are created and matching files are moved.
T5	Include duplicate file names in a destination category.	Existing files are not overwritten by duplicate names.
T6	Complete an organization run.	The application displays the number of files organized.
T7	Try a folder without files or with restricted access.	The application should handle the situation without data loss; exact behaviour should be verified during testing.

6.2 Test procedure

1. Prepare a test folder containing copies of sample files from several categories.
2. Launch the application and select the test folder using Browse.
3. Run the organization process and inspect the resulting category folders.
4. Check that files are placed in the expected categories.
5. Test duplicate file names and confirm that existing files are not overwritten.
6. Check the displayed organization count against the files processed.
7. Review any error messages and repeat tests with empty or inaccessible folders where safe.

6. PERFORMANCE TEST (CONTINUED)

6.3 Performance outcome

Area	Documented behaviour / evaluation
Folder selection	A Browse control allows a folder to be selected.
File categorization	Files are categorized by file type into predefined groups.
Folder creation	Category folders are created for organizing files.
File movement	Files are moved to the matching category folders.
Duplicate handling	The README states that duplicate file names are prevented from being overwritten.
Result display	The GUI displays the number of files organized.
Speed / scale	No verified runtime, throughput or large-folder benchmark is published in the repository.

Testing limitations

The repository README supports the feature descriptions above, but it does not provide measured performance results. Before wider use, test with different folder sizes, file permissions, duplicate names, unusual extensions and interrupted operations. Always test on copied sample data first to avoid accidentally moving important personal files.

7. MY LEARNINGS

The File Organizer project gave me practical experience in using Python to automate a task that users commonly perform manually. It helped me connect a graphical interface with file-system operations and understand the importance of separating user interaction from the core organization logic.

Technical learning

- Learned how a Tkinter interface can make a Python application easier to use.
- Improved understanding of file paths and operating-system interactions.
- Used the purpose of OS and Shutil modules in file management.
- Understood how file extensions can be mapped to predefined categories.
- Recognized why duplicate-name handling matters when moving files.
- Learned to document project structure and setup instructions in a README.
- Practised organizing code into separate application and organizer modules.

Problem-solving and professional learning

- Broke the file-organization task into selection, categorization, folder creation and movement stages.
- Recognized the importance of testing with representative sample files.
- Considered the risks of file operations and the need to prevent accidental overwrites.
- Used GitHub to make source code and project documentation accessible for review.

Career relevance

This project supports my development in Python programming and software problem-solving. It provides an example of how to automate a practical task, design a simple GUI and work with the file system. The experience is a foundation for learning more about software testing, exception handling, packaging and application design.

8. FUTURE WORK SCOPE

The current project provides a useful starting point for desktop file automation. The following are potential future improvements, not claims about features already present in the repository.

8.1 Usability improvements

- Add a preview mode that shows planned file movements before applying them.
- Provide clearer progress feedback for folders containing many files.
- Allow users to configure categories and file-extension rules.
- Add an option to choose a separate destination directory.
- Provide a summary report showing successful moves and skipped files.

8.2 Safety and reliability

- Improve error handling for permission issues, locked files and invalid paths.
- Offer a safe undo or restore feature with a record of file movements.
- Add automated tests for category mapping and duplicate-name handling.
- Handle files with no extension and uncommon file types consistently.
- Add confirmation steps before moving files in important directories.

8.3 Distribution and maintenance

- Document installation and execution steps in the README.
- Package the application for easier installation where appropriate.
- Test compatibility across supported operating systems.
- Maintain a clear dependency list and version-control changes.

CONCLUSION

File Organizer is a Python GUI application that sorts files into categories based on their file types. The documented features include folder selection, automatic categorization, creation of category folders, file movement, duplicate-name protection and a count of organized files. The project provides practical learning in Python, Tkinter and file-system operations. More extensive testing, configurable rules and safety features could further improve its reliability and usability.

— End of Report —